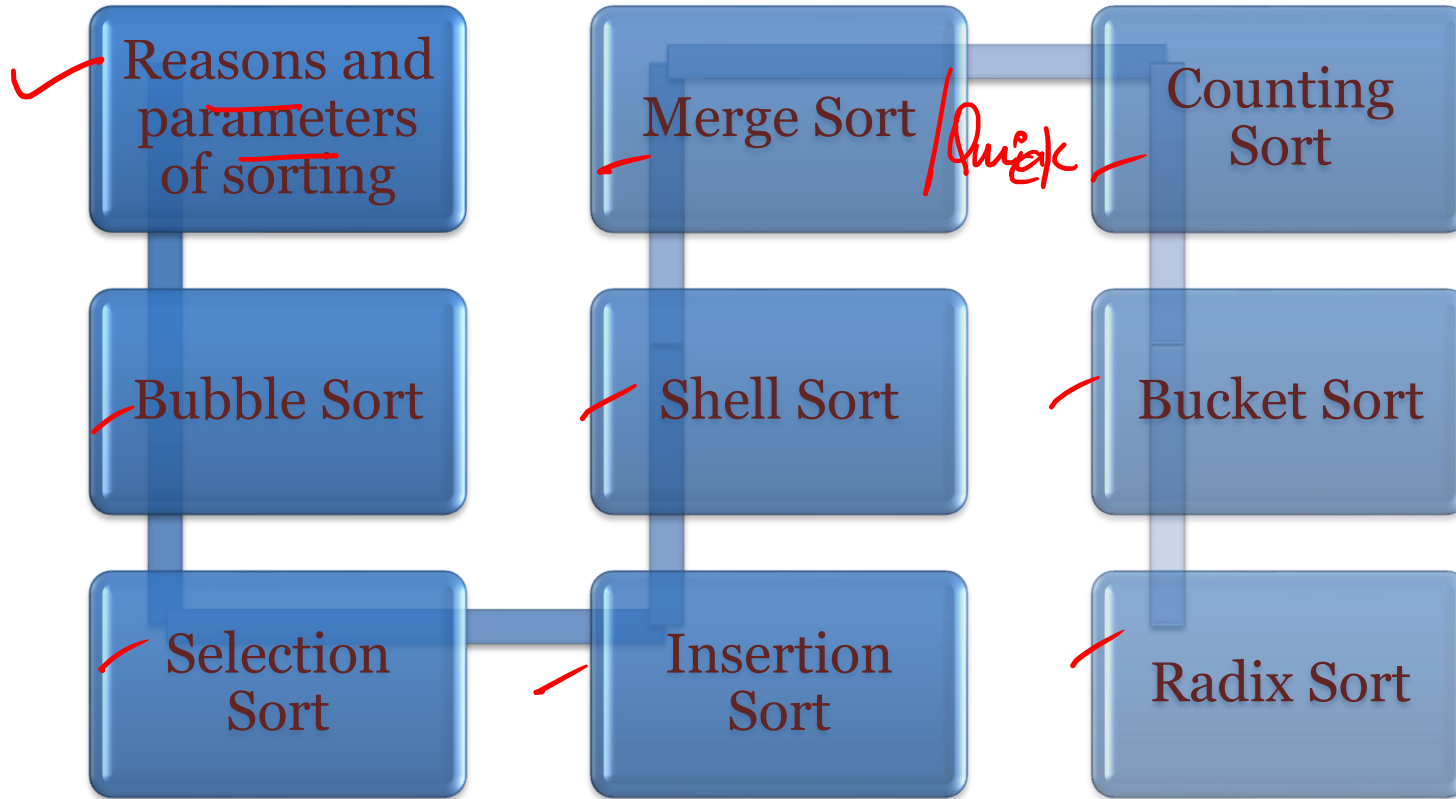


Sorting



Deliverables



Sorting

Given N nos. , rearrange them so that

✓ $a_1 \leq a_2 \leq a_3 \leq a_4 \leq a_5 \leq a_6 \leq a_7 \leq a_8 \dots \leq a_{n-1} \leq a_n$ ✓

There are $n!$ different permutations possible for this, and only one is the right sequence

Why Sorting

Sorting solves togetherness problem
or to find the similar elements

Matching items in two or more files

Searching by key values (Binary Search)

Comparison is the key

✓ Date comparison

```
CompareDate(A,B){
```

```
    → if (A.year < B.year) return -1;  
    → if (A.year > B.year) return +1;  
    → if (A.Month < B.Month) return -1;  
      if (A.Month > B.Month) return +1;  
      if (A.Day < B.Day) return -1;  
      if (A.Day > B.Day) return +1;
```

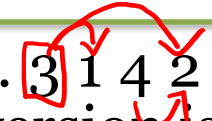
```
}
```

Single date comparison is doing 6 integer comparisons. Similarly we should think of file names, Domain names

Inversions and Runs

How many numbers are in reverse order

Inversions of a permutation: if $a_i > a_j$ and $i < j$ the pair (a_i, a_j) is called the inversion of the permutation.

e.g.  3 1 4 2 has three inversion namely (3,1) (3,2) and (4,2). Each inversion is a pair of elements that are out of sort.

✓ Runs in a given list represent sorted segments of the data in the said order

The sequence  has four runs

✓Tableaux

- An array of integers with left justified rows is called tableaux such that entries in each row are in increasing order from left to right and entries in each column are increasing from top to bottom.

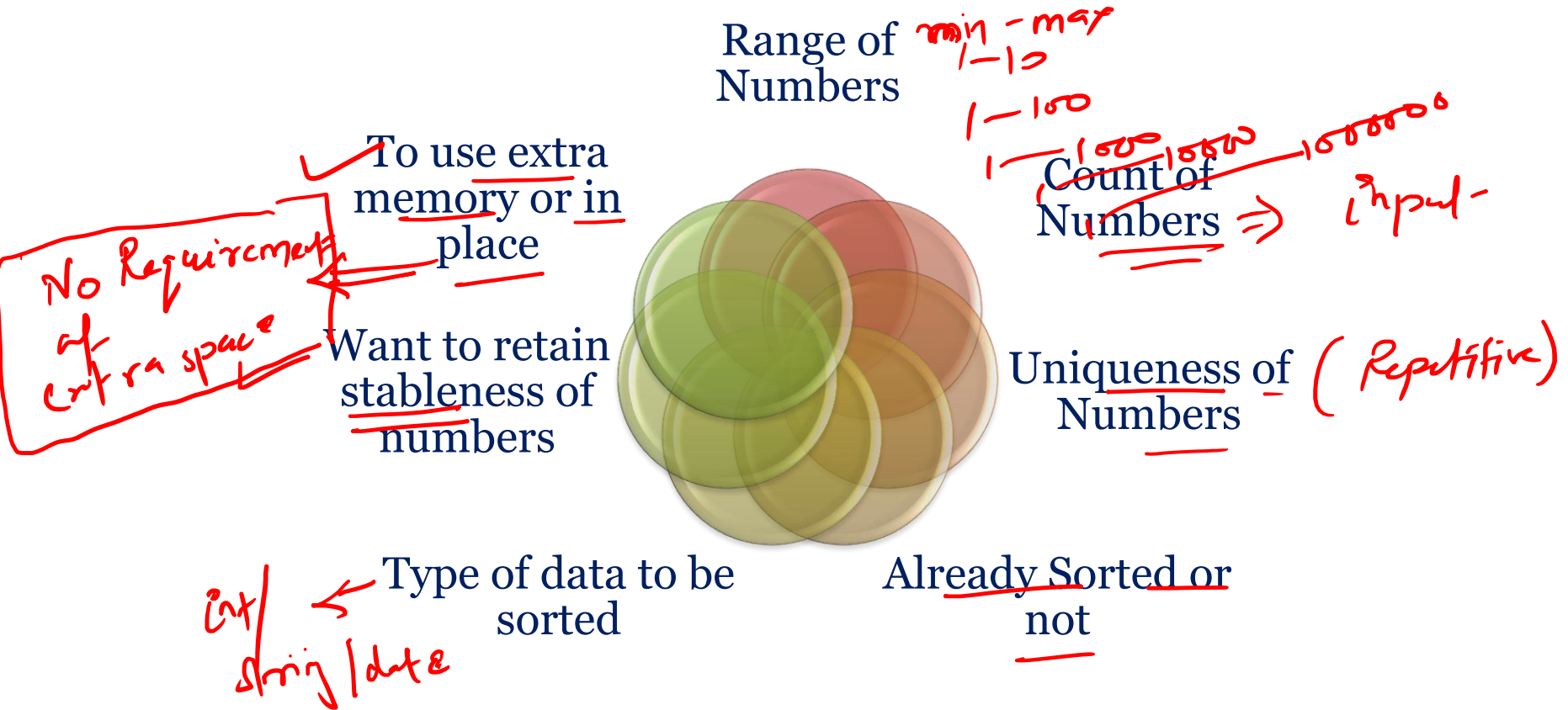
smaller

greater

larger

1	2	5	9	10	15
3	6	7	13		
4	8	12	14		
11					

Factors that may affect sorting



Stable and unstable sort

↓
order will not
change for duplicate
elements

Ram	3
Krishan	4
Shyam	5
Narhari	8
Vaman	4
Kapil	5
Budha	4
Balram	8

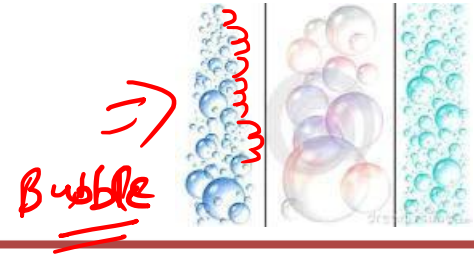
Ram	3
Krishan	4
Vaman	4
Budha	4
Shyam	5
Kapil	5
Narhari	8
Balram	8

- Stable

unstable

Ram	3
Budha	4
Krishan	4
Vaman	4
Shyam	5
Kapil	5
Balram	8
Narhari	8

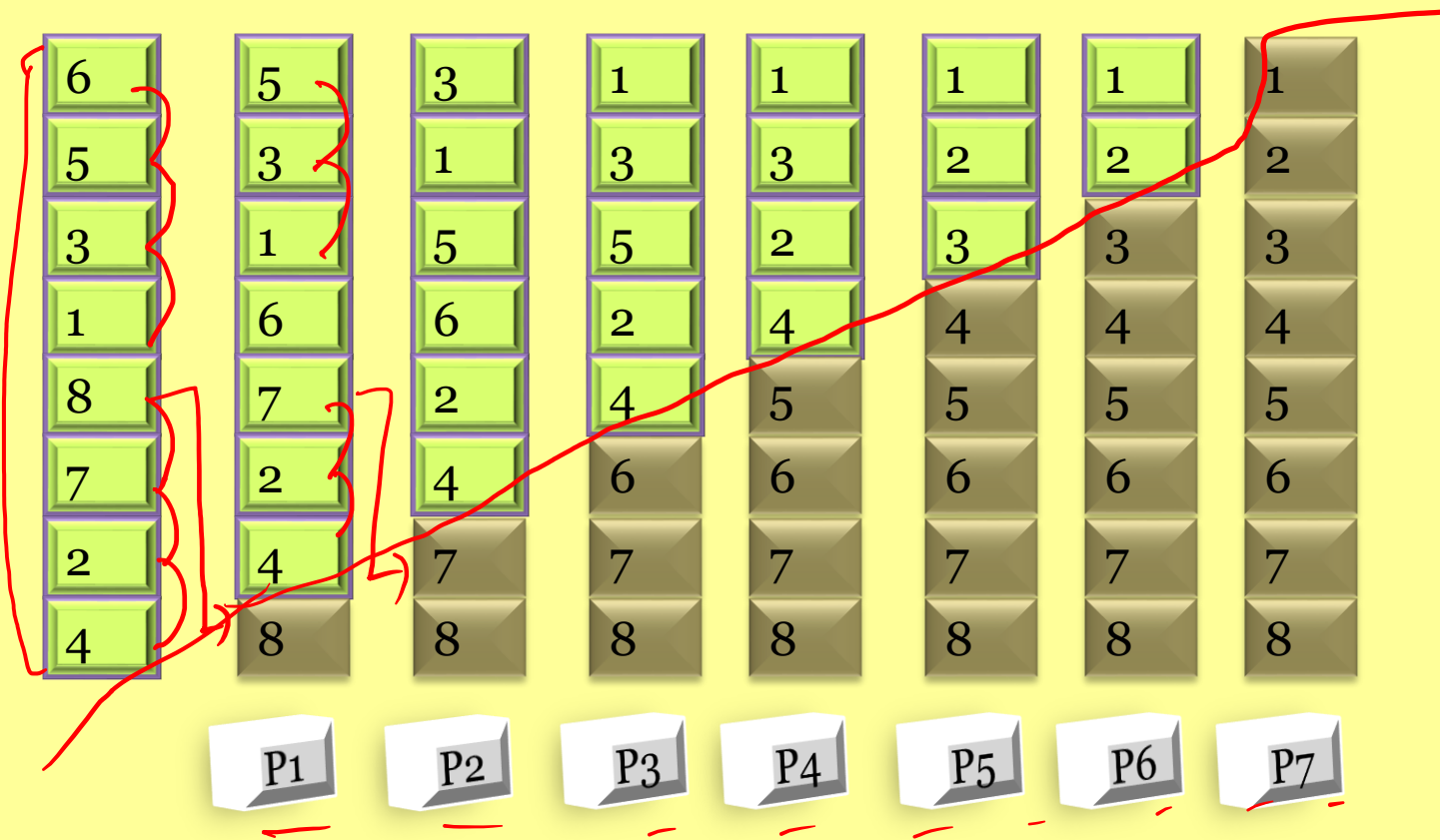
Bubble Sort



It works by comparing adjacent elements of an array and exchanges them if they are not in order.

After each iteration (pass) the largest element sinks to the **last** position of the array. In next iteration, second largest element sinks to second last position and so on

```
✓ for i = 1 to n-1 do
    for j = 1 to n-i do
        If (a[j+1] < a[j]) then swap(a[j] and a[j+1])
```



Bubble Sort

Takes relatively less swaps for sorting the nearly sorted data

Slowest algorithm for sorting nearly reverse sorted data

Speed in which largest element sinks is high - it occupies its proper place in one pass

Speed in which small element bubbles up is slow- only one exchange is done on each path

Largest elements are rabbits and smallest tortoises

It is stable, in place and popular due to tiny code

Order will not change. No Extra Computation (Space)

Bubble sort-improvements

Worst case and average case complexity is $O(n^2)$

Best case complexity is $O(n)$ ✓

Traversing in opposite direction in alternate passes; then it is bidirectional bubble, cocktail or shaker sort.

If there are no exchanges, then the sorting should be complete

Each next pass ends at the last swap of the previous pass

Recurrence

$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + (n-1)$$

$$T(n) = T(n-2) + n + (n-1)$$

$$T(n) = T(n-3) + n + (n-1) + (n-2)$$

·
·
·

$$T(n) = T(1) + n + (n-1) + (n-2) + \dots + 3 + 2$$
$$= \boxed{n(n+1)/2} = O(n^2)$$

$$\text{and } T(1) = 0$$

Selection Sort



Selection sort works by selecting the maximum from the given sequence and **swapping** it with last element in the list. Then again selecting maximum element out of the first $n-1$ elements and swapping it with the second last element and so on.

✓ For $i = 1$ to $n-1$ do

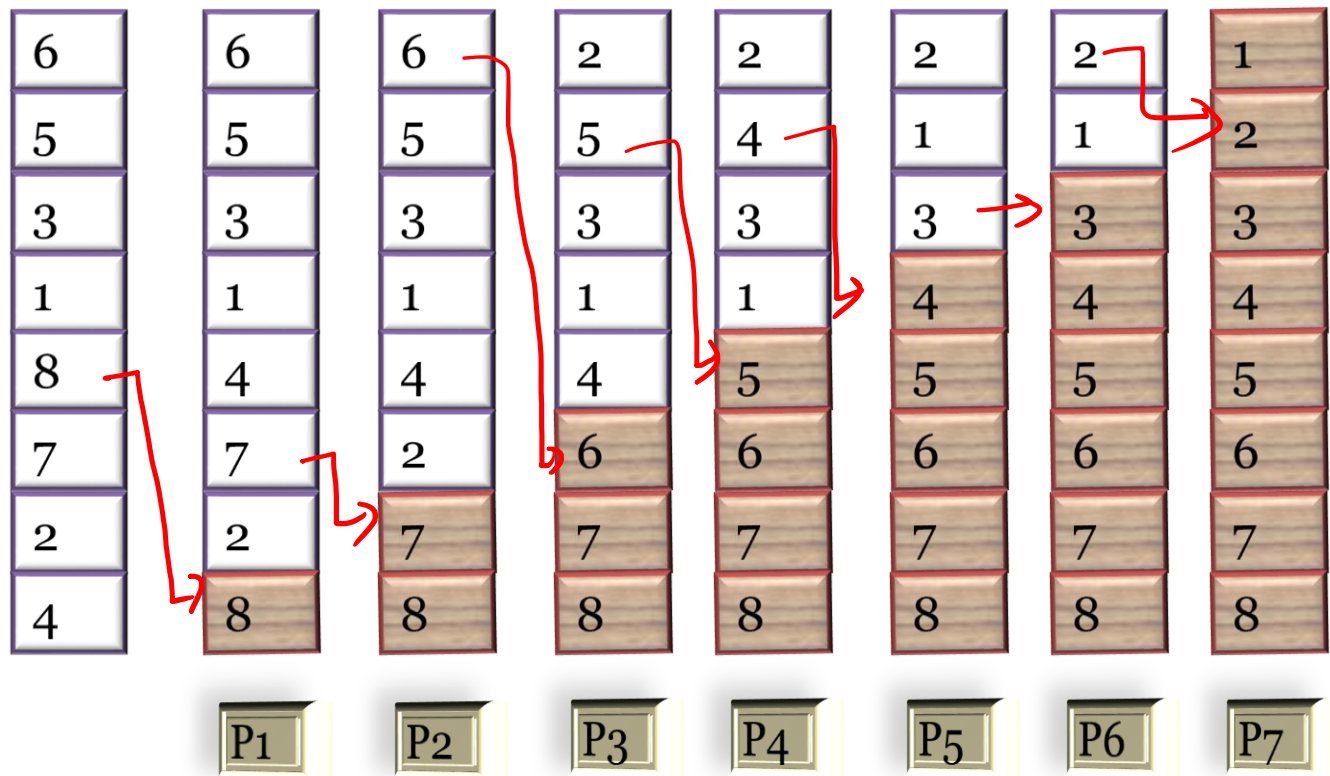
set min = i ✓

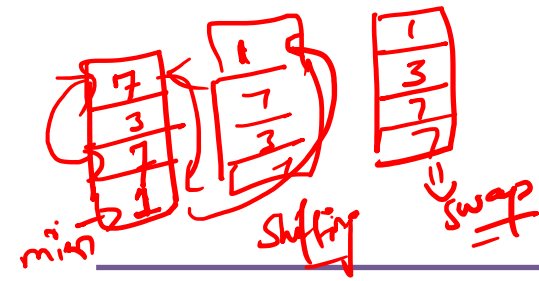
For $j = i+1$ to n do

If ($a[j] < a[\text{min}]$) then set min = j

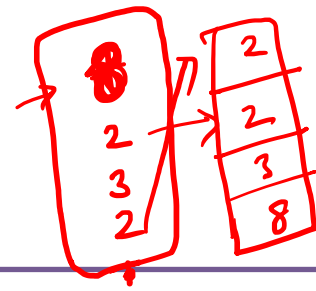
If ($i < \text{min}$) then swap $a[i]$ and $a[\text{min}]$

min





Selection Sort



Complexity is $O(n^2)$ for best, average and worst case.

Stable and in place sorting algorithm. It is stable if we bring the minimum value on the top and then shift all the elements down. If we swap the minimum element with the first position then it is not stable.

Min max can be applied.

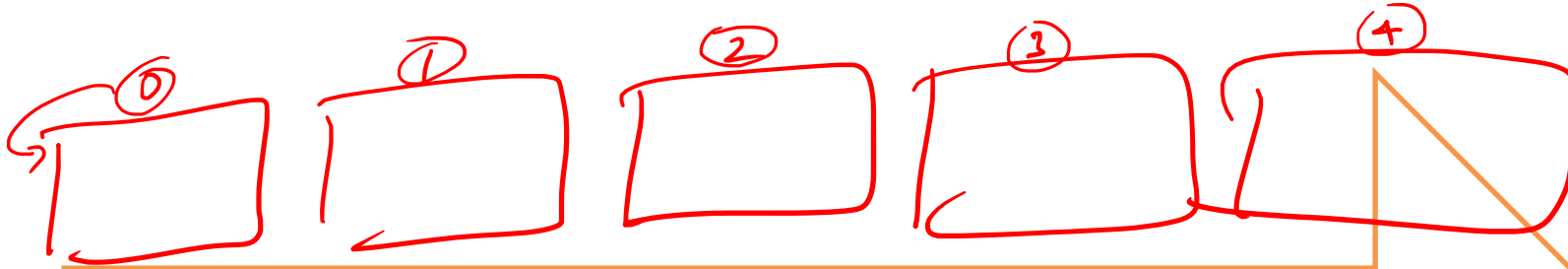
It outperforms bubble sort.

Number of swap operations is very less (Linear).

Independent of the initial ordering of the elements.

As n increases, selection sort keeps slowing down.

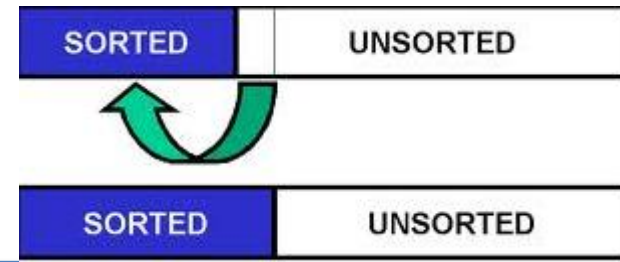
Key points



Movements are less. So when we have heavy file of records to be sorted based on some small index, then this type of sort may be useful. e.g. To reorganize your video files or music files, because cost of comparison will be **negligible** in terms of data movement.

Insertion Sort

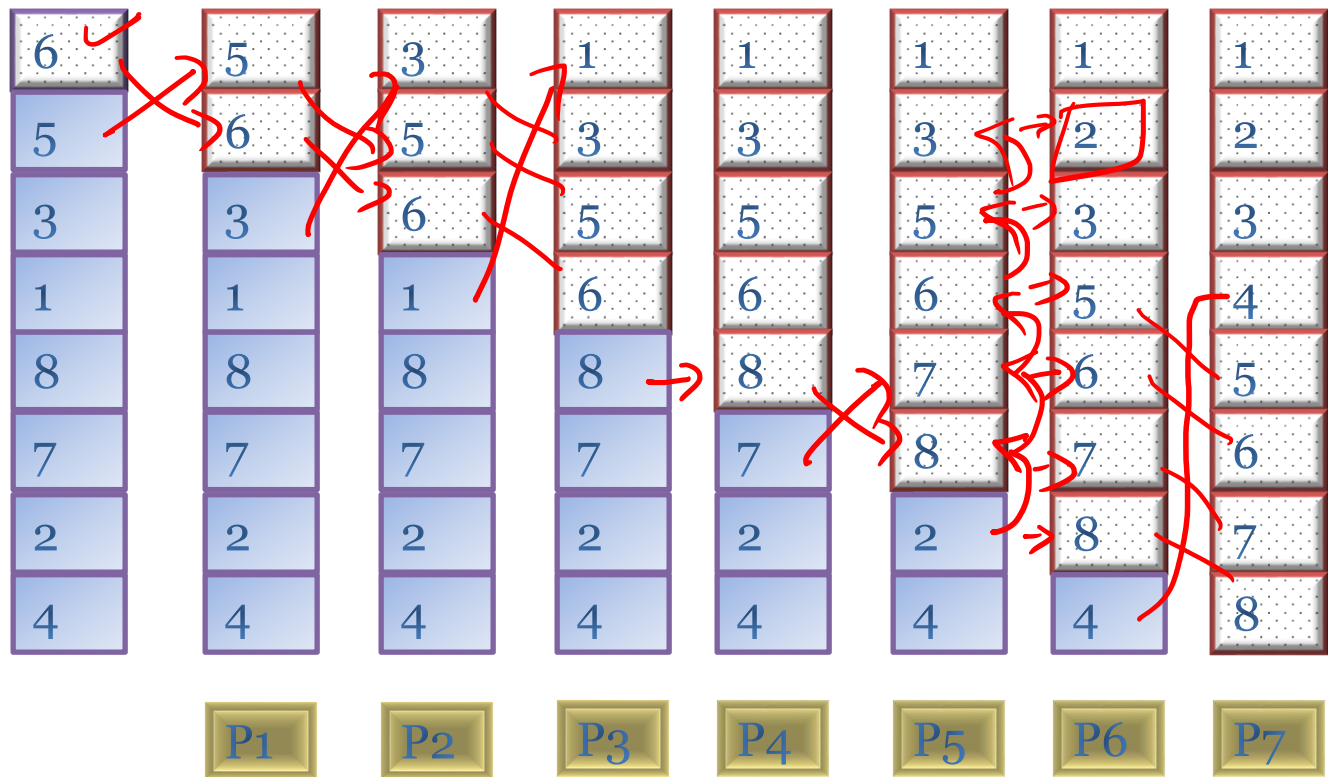
(Card's arranging)



It Considers the numbers one by one from the start. If it is a single number then it is sorted by default.

Then we consider the next number and put it at a proper position and now the list of two is sorted. Then we consider the third number and so on.

Before examining record R_j , we assume that the preceding records R_1 to R_{j-1} have already been sorted and we insert R_j into its proper place among the previously sorted records.



Insertion Sort Algorithm

Insertion Sort(A)

```
1.   for j ← 2 to length[A]
2.       do key ← A[j] //insert A[j] into the sorted sequence A[1..j-1]
3.       i ← j-1
4.       while i > 0 and A[i] > key
5.           do A[i+1] ← A[i]
6.           i ← i-1
7.       A[i+1] = key
```

Insertion-Sort invariant: At the start of each iteration of the for loop of lines 1-7 the sub array $A[1..j-1]$ is the sorted version of the original elements of A.

Insertion Sort-improvements

Binary Search with in sorted sequence

✓ $O(n^2)$ in worst and average case. Inserting in between is costly because we have to move remaining elements.

Good algorithm for small list up to 25-30 numbers. It is stable and in place. It outperforms bubble and selection.

Best case $O(n)$. Works well for nearly sorted lists.

Good for cases where whole data is not available initially and new data values keep on adding in the list

Key points

Number of shifts is equal to the number of inversions and number of comparisons is (no of inversions + n - 1)

An array is partially sorted if the number of inversions is $O(N)$.

A small array appended to a large sorted array.

An array with only a few elements out of place.

Shell Sort (Extension of Insertion)

If we have a sorting algorithm which moves items only one position at a time, its average running time will be at best proportional to n^2 .

For substantial improvements over straight insertion, we need a mechanism by which records can take long leaps instead of short steps.

One implementation of shell sort is arranging the data sequence in a two-dimensional array and then sorting the columns of array using insertion sort.

25 elements - 5 columns
↓
Insertion Sort

Shell Sort

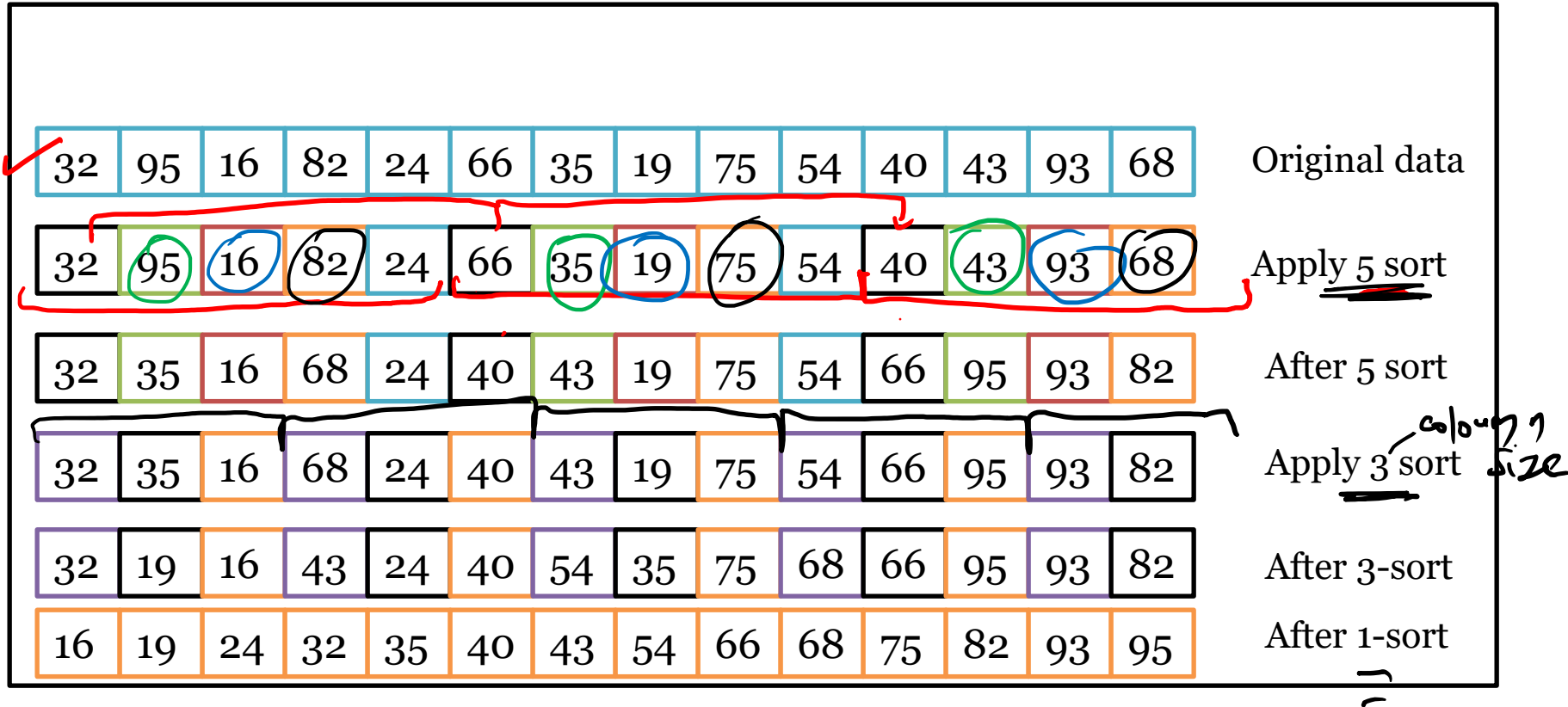
Sort sequence can be 16-sort, 8-sort, 4-sort, 2-sort.
chain size

This does not allow the interaction between even and odd keys, so we can try 7-sort, 5-sort, 3-sort and 1-sort

Shell Sort performs better than bubble, selection or insertion sort.

The Worst case complexity is $O(n \log^2 n)$ or $O(n^{3/2})$ or others depending upon the Gap sequence.
 $\sqrt{O(n^2)}$

Shell Sort Example



Insertion sort will take 38 swaps while Shell sort is taking 26 swaps

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	95	16	82	24	66	35	19	75	54	40	43	93	68
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

6 swaps

32	35	16	68	24	40	43	19	75	54	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

32	19	16	43	24	40	54	35	75	68	66	95	93	82
----	----	----	----	----	----	----	----	----	----	----	----	----	----

5 swaps

16	19	24	32	35	40	43	54	66	68	75	82	93	95
----	----	----	----	----	----	----	----	----	----	----	----	----	----

15 swaps